# CNNs, CNN Architectures, Augmentations

Sattam Altwaim
satttam.tuuaim@kaust.edu.sa



جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology



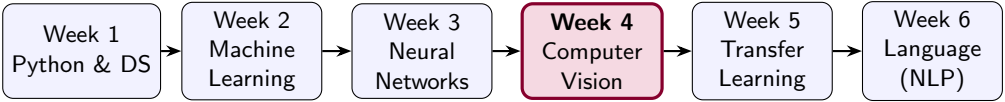

KAUST Academy & Mawhiba
AI Olympiad Training (Beginners)
Week 4, Day 23, Lecture

11 October 2026

# Table of Contents


1. Big Picture and Motivation
2. Learning Outcomes
3. The CNN Recipe: Blocks and Stages
4. CNN Architectures
5. Data Augmentation
6. Olympiad Practice
7. Limitations and What Comes Next
8. Summary
9. References

# Where We Are




**Today: Day 23 of 42**

- **Yesterday:** a convolution is a small shared neuron slid over the image; padding, stride and pooling set the shapes; two blocks of conv, ReLU, pool and a `Linear` made a CNN, written as a class and trained with Week 3's loop.
- **Today:** how those blocks are stacked into the networks that won ImageNet (LeNet, AlexNet, VGG, ResNet); the two layers that make deep stacks trainable (`BatchNorm2d`, the skip connection); global average pooling; and **data augmentation**, the cheapest way to get more training images.
- **This week:** tomorrow, detection and segmentation; then shapes and classification in depth (Day 25), YOLO and U-Net (Day 26), augmentation techniques and vision habits (Day 27), and Friday.

# How We Did It Before: Shallow Nets and Hand-Made Features

- **Day 22's CNN:** two conv blocks and a `Linear`, 20,490 parameters. It is LeNet-5's shape (1998): two convolutions, 60,000 parameters, 0.95% error on MNIST.
- **Photos before 2012:** the ImageNet challenge (1,000 classes, 1.2 million photos) was won with **hand-made features** (SIFT, Fisher vectors) and a linear classifier: Week 2's recipe at scale.
- The 2011 winner's top-5 error: **25.8%** (Russakovsky et al., 2015).

| Year | Winner | Top-5 error |
|---|---|---|
| 2010 | hand-made features + linear | 28.2% |
| 2011 | hand-made features + linear | 25.8% |
| 2012 | AlexNet, 8 layers | 16.4% |
| 2014 | VGG, 16 layers | 7.3% |
| 2014 | GoogLeNet, 22 layers | 6.7% |
| 2015 | ResNet, 152 layers | 3.57% |
| | a trained human | about 5.1% |

Sources: Russakovsky et al. (2015) for 2010 to 2014 and the human estimate; Simonyan and Zisserman (2015) Table 7 for VGG; He et al. (2016) for ResNet. Top-5 error: the correct label is not among the network's five guesses.

Every row after 2011 is Day 22's block stacked **deeper**; today is about what had to be invented for that stacking to work.

# Where It Breaks

Three problems with "just stack more blocks"

- **Deeper plain nets train worse.** He et al. (2016, Fig. 1) stacked 20 and then 56 plain conv layers on the same data: the 56-layer net had *higher training error*, not just higher test error. Not overfitting: the optimiser could not even fit the training set. They called it the **degradation problem**.
- **Training is slow and fragile.** Each layer's inputs shift as the layers before it change, so learning rates must be small and starting values careful. Ioffe and Szegedy (2015) measured the fix: with batch normalization the same accuracy in **14 times fewer** training steps.
- **More parameters, same photos.** AlexNet's 60 million parameters on 1.2 million photos: "without this scheme, our network suffers from substantial overfitting" (Krizhevsky et al., 2012), the scheme being random crops and flips of every training image. LeNet-5 went from 0.95% to 0.8% error by training on shifted and distorted copies (LeCun et al., 1998).

The pattern

Depth needs two helpers inside the network (normalise, and let the input skip ahead), and small data needs help outside it (make every photo count several times).

# Today's Idea

## In one sentence

A modern CNN is Day 22's block repeated in **stages** (conv $3 \times 3$, `BatchNorm2d`, ReLU; pool between stages; channels doubling), with a **skip connection** around each pair of convolutions and **global average pooling** at the end; and it is trained on images that are randomly flipped, cropped and recoloured every epoch, **data augmentation**.

- **BatchNorm2d** is Day 17's BatchNorm per channel: it keeps every layer's inputs on the same scale, so deep stacks train fast at a large learning rate.
- **The skip connection** adds the block's input to its output, $y = F(x) + x$: a block that has nothing to add can pass $x$ through untouched, and the gradient has a straight road back.
- **Augmentation** changes the training transform only: validation and test images stay as they are.

## Where you will meet it

Every Kaggle or IOAI image task: from scratch, a small VGG-style network with BatchNorm (today's labs); from a pretrained ResNet when the photos are real and few (Week 5). Augmentation is in both, and Day 27 is a whole deck of its techniques.

# ⇒ Learning Outcomes

By the end of this session, you will be able to:

- **Describe** the block-and-stage recipe of a CNN (conv, BatchNorm, ReLU; pool; channels doubling) and compute how far into the image a deep unit sees.
- **Explain** what LeNet-5, AlexNet, VGG and ResNet each added, with their published sizes and ImageNet errors.
- **Explain** `BatchNorm2d` and the skip connection, and why they make deep networks trainable.
- **Replace** `Flatten` and a large `Linear` by global average pooling, and count the parameters saved.
- **Choose** data augmentations that keep the label, put them in the training transform only, and read their effect on the training-against-validation curves.
- **Build** a VGG-style model class from a block function and train it, with augmentation, on a Kaggle image set.

# CNNs: **The Recipe: Blocks and Stages**

# Definition: Block, Stage, Head

## Definition

A **block** is `Conv2d(3 × 3, padding 1)` → `BatchNorm2d` → ReLU. A **stage** is one to three blocks at the same width followed by `MaxPool2d(2)`. A CNN is three to five stages whose channel widths double, then a **head**: global average pooling and one `Linear` to the classes.




Day 22's network was one block per stage without BatchNorm and with `Flatten` plus a big `Linear` as the head. Today's lab network is the picture above: VGG's shape at a tenth of the size.

# Why Small Kernels Stacked: The Receptive Field

### Definition

The **receptive field** of a unit is the patch of the input that can change its value. One $3 \times 3$ conv sees $3 \times 3$; each further one adds 2 pixels per side at the current step; a pool doubles the step.

$$r_\ell = r_{\ell-1} + (k-1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1} \cdot s_\ell$$

$r$: receptive field in input pixels; $j$: the step between neighbouring units; $k$: kernel size; $s$: stride (2 for a pool); $r_0 = j_0 = 1$.

- Two $3 \times 3$ convs see $5 \times 5$ with $2 \cdot 9 = 18$ weights per channel pair and two ReLUs; one $5 \times 5$ conv needs 25 for one ReLU. Three $3 \times 3$: $7 \times 7$ with 27 against 49.

| Layer | step $j$ | field $r$ |
|---|---|---|
| conv $3 \times 3$ | 1 | 3 |
| conv $3 \times 3$ | 1 | 5 |
| pool 2 | 2 | 6 |
| conv $3 \times 3$ | 2 | 10 |
| conv $3 \times 3$ | 2 | 14 |
| pool 2 | 4 | 16 |
| conv $3 \times 3$ | 4 | 24 |
| conv $3 \times 3$ | 4 | 32 |
| pool 2 | 8 | 36 |

One row checked: after the first pool $r = 5 + 1 \cdot 1 = 6$, $j = 2$; the next conv gives $r = 6 + 2 \cdot 2 = 10$. After three stages a unit sees $36 \times 36$ of a $64 \times 64$ photo: a whole object, not an edge.

# In Code: A Block Function and a Model Class

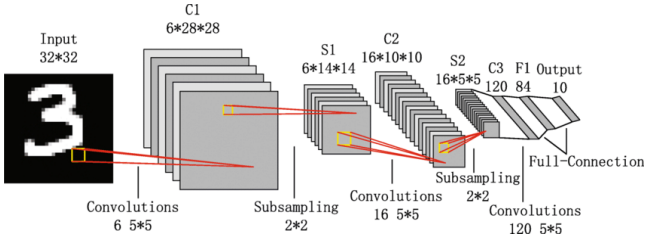
```
def block(c_in, c_out):                          # conv, BatchNorm, ReLU
    return nn.Sequential(nn.Conv2d(c_in, c_out, 3, padding=1),
                         nn.BatchNorm2d(c_out), nn.ReLU())

class VGGSmall(nn.Module):
    def __init__(self, n_classes=6):
        super().__init__()
        self.features = nn.Sequential(
            block(3, 32), block(32, 32), nn.MaxPool2d(2),        # (32, 32, 32)
            block(32, 64), block(64, 64), nn.MaxPool2d(2),       # (64, 16, 16)
            block(64, 128), block(128, 128), nn.MaxPool2d(2))    # (128, 8, 8)
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
                                  nn.Linear(128, n_classes))
    def forward(self, x):
        return self.head(self.features(x))
model = VGGSmall()
print(model(torch.zeros(1, 3, 64, 64)).shape)        # (1, 6)
print(sum(p.numel() for p in model.parameters()))    # 288678
```

- The same class shape as yesterday; the new pieces are `block`, `BatchNorm2d` and the head's `AdaptiveAvgPool2d(1)`, each explained in the next section. Six convolutions, 288,678 parameters, and the head's `Linear` has only $128 \cdot 6 + 6 = 774$ of them.

# CNNs: **The Architectures That Won ImageNet**

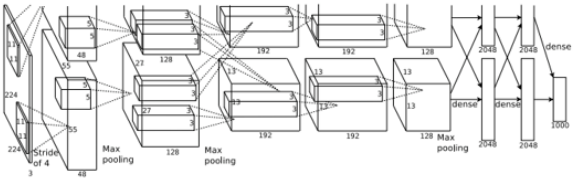
# LeNet-5 (1998): The Recipe Is Already There




Source: redrawn after LeCun, Bottou, Bengio and Haffner (1998), Fig. 2; layer names as in the paper (C: convolution, S: subsampling, F: fully connected).

- Input $32 \times 32$. C1: six $5 \times 5$ kernels, $6 \times 28 \times 28$. S2: average pooling to $6 \times 14 \times 14$. C3: sixteen kernels, $16 \times 10 \times 10$. S4: pool to $16 \times 5 \times 5$. Then 120, 84 and 10 units. About **60,000** parameters; **0.95%** test error on MNIST, 0.8% with distorted copies.
- Conv, pool, conv, pool, then a classifier: Day 22's pipeline, drawn 28 years ago. What changed since is only the depth, the width, and three helpers.

# AlexNet (2012): Depth, GPUs, ReLU, Dropout, Augmentation




Source: Krizhevsky, Sutskever and Hinton (2012), Fig. 2: the network split across two GPUs; numbers are channel counts and map sizes.

- $224 \times 224 \times 3$ in; five convolutions (the first $11 \times 11$ with stride 4 and 96 kernels, then $5 \times 5$, then three $3 \times 3$), three max pools, three fully connected layers of 4,096, 4,096 and 1,000 units. **60 million** parameters.
- Four things made it trainable: **ReLU** instead of sigmoid (Day 15), **dropout** 0.5 in the fully connected layers (Day 17), two **GPUs** for a week, and **augmentation**: random 224 crops of 256 images plus flips, 2,048 versions of every photo.
- ImageNet 2012: **16.4%** top-5 error against 25.8% the year before (Russakovsky et al., 2015). Deep learning's first headline.

# VGG (2014): Only $3 \times 3$, Just Deeper

- Simonyan and Zisserman asked one question: what if every kernel is $3 \times 3$ and the net is simply deeper? **16 to 19** weight layers, widths **64, 128, 256, 512, 512** doubling after each of five pools, then the AlexNet head.
- Two stacked $3 \times 3$ convolutions see what one $5 \times 5$ sees, with fewer weights and an extra ReLU (the receptive-field frame).
- **138 million** parameters for VGG-16, 124 million of them in the three fully connected layers. ImageNet 2014: **7.3%** top-5 error, second place behind GoogLeNet's 6.7%.
- Its shape, block $\times n$ then pool with doubling widths, is what "a small CNN" means today, and today's lab network.

| Stage | Layers (VGG-16) | Map |
|---|---|---|
| 1 | conv 64, conv 64, pool | $112 \times 112$ |
| 2 | conv 128, conv 128, pool | $56 \times 56$ |
| 3 | conv 256 $\times 3$, pool | $28 \times 28$ |
| 4 | conv 512 $\times 3$, pool | $14 \times 14$ |
| 5 | conv 512 $\times 3$, pool | $7 \times 7$ |
| head | FC 4096, FC 4096, FC 1000 | |

Configuration D of Simonyan and Zisserman (2015), Table 1; input $224 \times 224$, every conv $3 \times 3$ with padding 1.

# Where Depth Breaks: The Degradation Problem




Source: Stanford CS231n lecture slides, sketched after He, Zhang, Ren and Sun (2016), Fig. 1: 20- and 56-layer *plain* networks on CIFAR-10.

## What the figure says

The 56-layer plain network is worse than the 20-layer one *on the training set*. A deeper net could copy the shallow one and pass the rest through unchanged, so a better solution exists; gradient descent does not find it.

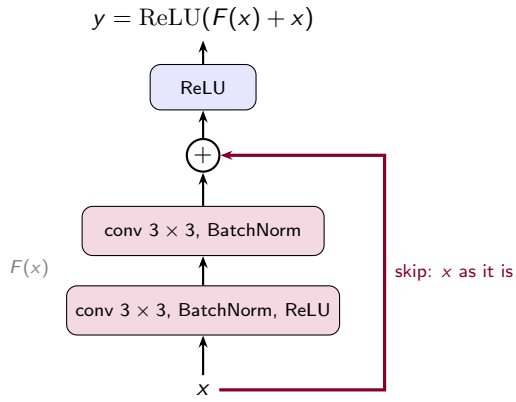
- Not overfitting (training error is the one that rose), and BatchNorm alone does not cure it (He et al., 2016, Section 4.1).
- The missing piece: make "pass it through unchanged" the *easy* thing for a block to learn. That is the skip connection.

# ResNet (2015): The Skip Connection

Definition

A **residual block** computes two convolutions $F(x)$ and adds its own input: $y = \mathrm{ReLU}\big(F(x) + x\big)$. The block learns what to *change*, the residual, not the whole mapping.

- **Why depth now works:** with all weights at zero, $F(x) = 0$ and $y = x$: the block is invisible, so 56 blocks are never worse than 20. And $\partial y/\partial x = \partial F/\partial x + 1$: the $+1$ carries the gradient straight back.
- **Numbers:** 152 layers, **3.57%** top-5 (ensemble; He et al., 2016). ResNet-18: 11.7 million parameters, ResNet-50: 25.6 million, far below VGG-16, because the head is global average pooling.



# BatchNorm2d: Day 17's BatchNorm, Per Channel

### Definition

`BatchNorm2d(C)` standardises each channel over the batch *and* its positions, then rescales it with two learned numbers per channel, $\gamma_c$ and $\beta_c$:

$$\hat{x} = \frac{x - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \qquad y = \gamma_c \hat{x} + \beta_c$$

$\mu_c, \sigma_c^2$: mean and variance of channel $c$ over (batch, height, width); $\epsilon = 10^{-5}$ keeps the division safe.
`BatchNorm2d(16)` has $2 \cdot 16 = 32$ parameters.
**With numbers:** one channel, values $1, 3, 5, 7$ at four positions: $\mu = 4$, $\sigma^2 = 5$,
$\hat{x} = (-1.34, -0.45, 0.45, 1.34)$; with $\gamma = 1$, $\beta = 0$ these are the outputs.

- **Where:** after the conv, before the ReLU, in every block; $\beta$ replaces the conv's bias.
- **What it buys** (Ioffe and Szegedy, 2015): the same accuracy in 14 times fewer steps, larger learning rates, less need for dropout.
- **Two modes** (Day 17): `train()` uses the batch's $\mu, \sigma$; `eval()` uses running averages. Forgetting `eval()` before validation is the most common BatchNorm bug.
- **Small batches** (below about 16) give noisy $\mu, \sigma$: keep 32 to 128.

# Global Average Pooling: A Head Without the Big Linear

Definition

**Global average pooling** replaces each channel's map by its mean: $(C, H, W) \to (C,)$, "how much of this pattern is in the picture". Then one `Linear(C, k)`.

- **Parameters:** Day 22's head on a $(64, 8, 8)$ map, `Linear(4096, 6)`: 24,582. GAP head, `Linear(64, 6)`: 390. VGG-16's FC layers hold 124 of its 138 million; ResNet-50 has 25.6 million in total.
- **Any image size:** the mean exists for $8 \times 8$ and $12 \times 12$ maps alike: one network for 64 and 96 pixel photos.
- Lin, Chen and Yan (2014); every network since ResNet ends this way.

```
fmap = torch.randn(2, 64, 8, 8)
gap = nn.AdaptiveAvgPool2d(1)
y = gap(fmap)
print(y.shape)              # (2,64,1,1)
print(y.flatten(1).shape)   # (2, 64)
head = nn.Sequential(
    nn.AdaptiveAvgPool2d(1),
    nn.Flatten(), nn.Linear(64, 6))
n = sum(p.numel()
        for p in head.parameters())
print(n)                    # 390
```

`AdaptiveAvgPool2d(1)` means "pool to a $1 \times 1$ map", whatever the input size; `fmap.mean(dim=(2, 3))` is the same in one line.

# The Ladder, With Numbers

| Network | Year | Weight layers | Parameters | ImageNet top-5 | What it added |
|---|---|---|---|---|---|
| LeNet-5 | 1998 | 5 | 60 thousand | (MNIST 0.95%) | conv, pool, classifier |
| AlexNet | 2012 | 8 | 60 million | 16.4% | ReLU, dropout, GPUs, augmenta |
| VGG-16 | 2014 | 16 | 138 million | 7.3% | only $3 \times 3$, stages with doubling v |
| GoogLeNet | 2014 | 22 | 6.6 million | 6.7% | parallel kernel sizes, no big FC he |
| ResNet-152 | 2015 | 152 | 60 million | 3.57% | skip connections, BatchNorm, GA |

Sources: LeCun et al. (1998); Krizhevsky et al. (2012); Simonyan and Zisserman (2015); Szegedy et al. (2015); He et al. (2016); errors as reported by Russakovsky et al. (2015) and the papers; parameter counts from the papers and torchvision's model tables.

- Read the columns together: depth went from 8 to 152 while the parameter count went *down* from VGG to ResNet, because the fully connected head disappeared. Depth, not size, bought the accuracy; BatchNorm and skips made the depth trainable.
- After 2015 the names you will meet are **MobileNet** (Howard et al., 2017; small enough for phones) and **EfficientNet** (Tan and Le, 2019; depth, width and image size grown together). Both are ResNet-style stacks of blocks.

# Which One to Use: Common Practice

| Situation | What people start from | Source and catch |
|---|---|---|
| Small images (28 to 96 px), thousands per class, must train from scratch | a VGG-style stack: 3 stages of 2 blocks, widths 32-64-128, BatchNorm, GAP head; Adam $10^{-3}$, 10 to 30 epochs | Simonyan and Zisserman 2015; today's lab. Catch: time one epoch before choosing the width |
| Real photos, hundreds to thousands per class | a pretrained ResNet-18 or ResNet-50, fine-tuned | He et al. 2016; Week 5. Catch: resize to the size it was trained at (224) |
| Must run on a phone or a CPU, or a tight time limit | MobileNet or EfficientNet-B0, pretrained | Howard et al. 2017; Tan and Le 2019 |
| Deeper than about 10 convolutions, any data | skip connections, always; BatchNorm in every block | He et al. 2016: plain nets degrade past that depth |
| Few images per class (tens) | do not train from scratch; pretrained features plus a small head (Week 5), with augmentation | Day 20's recipe card |

Starting points, as on Day 18 and Day 22: begin here, read the train-against-validation curves, then widen, deepen, regularise or augment one thing at a time.

# CNNs: **Data Augmentation**

# Definition: More Training Images for Free

### Definition

**Data augmentation** applies a *random*, *label-preserving* change to a training image every time it is loaded: a flip, a crop, a rotation, a colour shift. The network never sees exactly the same picture twice, and the label stays what it was.

- **Why it works:** Day 22's convolution tolerates shifts, nothing else. A rotated, resized or darker object is a new pattern for every kernel; augmentation shows the network those versions *during training*, so it learns kernels that survive them.
- **Why it is needed:** hundreds of thousands of parameters memorise a few thousand photos (Day 17's gap). Augmentation is regularisation from the data side.
- **Published effect:** LeNet-5, 0.95% $\rightarrow$ 0.8% with shifted and distorted copies (LeCun et al., 1998). AlexNet's 2,048 crops and flips per photo: "without this scheme, our network suffers from substantial overfitting" (Krizhevsky et al., 2012).

### The test it must pass

An augmented image must be one that *could have been in the test set with the same label*. A flipped cat is a cat; a flipped 7 is nothing.

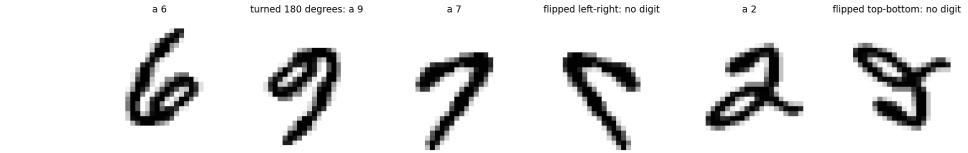
# The Catalogue on One Photo




Source: one photo of Kaggle's garbage-classification set (TrashNet, `glass/glass1.jpg`) under fixed `torchvision.transforms.functional` calls; `scripts/day23_cnn_figures.py`.

- **Geometric:** flips, rotation, random resized crop, shifts: what the convolution does not tolerate (orientation, scale, position). The black corners are the cost of rotating and shifting.
- **Photometric:** brightness, contrast, saturation (`ColorJitter`), grayscale: lighting and camera differences. **Erasing** a box teaches the network not to rely on one part.

# Label Safety: Which Transforms Are Allowed




Source: MNIST digits via OpenML, rotated and flipped with NumPy; `scripts/day23_cnn_figures.py`.

| Transform | Safe for | Not safe for |
|---|---|---|
| horizontal flip | animals, scenes, objects, satellite tiles | digits, letters, text, left/right labels |
| vertical flip, 180° | satellite and microscope images | anything with an up (faces, digits, street sc |
| rotation $\pm 15°$ | almost everything | tasks where tilt is the label |
| random resized crop | objects that fill the frame | tiny objects that a crop can cut out |
| colour jitter, grayscale | shape-defined classes | colour-defined classes (ripe fruit, flowers, tr |

The rule is the test of the definition frame: would this picture, with this label, be a fair test image?
Look at eight augmented samples before training (the lab does), and ask it for each.

# Where It Lives: The Training Transform Only

```
train_tf = transforms.Compose([
    transforms.RandomResizedCrop((96, 128), scale=(0.7, 1.0)),   # random box, resized
    transforms.RandomHorizontalFlip(),                            # probability 0.5
    transforms.RandomRotation(15),                                # -15 to 15 degrees
    transforms.ColorJitter(brightness=0.3, contrast=0.3, saturation=0.3),
    transforms.ToTensor()])
eval_tf = transforms.Compose([transforms.Resize((96, 128)), transforms.ToTensor()])
train_ds = datasets.ImageFolder(root, transform=train_tf)   # or a Dataset class
val_ds = datasets.ImageFolder(root, transform=eval_tf)      # fixed version only
```




Source: the same Kaggle photo through `train_tf` eight times, seed 0; `scripts/day23_cnn_figures.py`.

- The transform runs inside `__getitem__` (Day 19): every epoch draws new versions, the stored photo never changes. **Validation and test** get `eval_tf`, one fixed version, or their scores become noise.

# Reading the Effect on the Curves

| | without | with augmentation |
|---|---|---|
| training loss | falls to near 0 | higher, noisier |
| training accuracy | reaches 1.0 | stays below 1.0 |
| validation accuracy | peaks early, then flat | keeps rising, ends higher |
| the gap (Day 17) | wide | narrow |
| epochs needed | few | more |

Augmentation makes the *training* task harder on purpose. Judge it by the validation curve only; a training accuracy that no longer reaches 1.0 is the plan working.

When it hurts

Too strong (crops that cut the object out, 90° rotations of street scenes, heavy colour shifts on colour-defined classes): training loss stays high *and* validation falls: underfitting a task that no longer matches the test set. Weaken one transform at a time.

- **Later this week (Day 27):** mixing images (mixup, cutmix), augmenting at test time and averaging the predictions (test-time augmentation), and how to choose the strength from the curves.

# CNNs: **Olympiad Practice**

# Practice Task: Five Kinds of Flower

## Task

Kaggle's *Flowers Recognition* set: 4,317 photos in five folders, `daisy` (764), `dandelion` (1,052), `rose` (784), `sunflower` (733), `tulip` (984); photos of different sizes, about $320 \times 240$ to $500 \times 375$, taken by many people. A hidden test part of the same kind is scored by accuracy; training must be from scratch, no pretrained weights.

- **Question 1:** which of today's augmentations keep the label here, and which one would you leave out or weaken? Think about what tells a rose from a tulip.
- **Question 2:** the photos are resized to $96 \times 96$. Propose the network: how many stages, which widths, which head, and roughly how many parameters?
- **Question 3:** after 10 epochs the training accuracy is 0.99 and the validation accuracy 0.68. Name the problem and the two moves from today, in the order you would try them.

Source: Kaggle dataset `alxmamaev/flowers-recognition` (photos collected from Flickr, Google Images and Yandex Images); class names and counts are the dataset's own.

# Solution Idea and Code

**Recognise:** classification from scratch on a few thousand photos: a VGG-style stack with BatchNorm and a GAP head, trained with augmentation.

```
train_tf = transforms.Compose([
    transforms.RandomResizedCrop(96, scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(), transforms.RandomRotation(20),
    transforms.ColorJitter(brightness=0.2, contrast=0.2),   # no hue, no saturation
    transforms.ToTensor()])
eval_tf = transforms.Compose([transforms.Resize((96, 96)), transforms.ToTensor()])
model = VGGSmall(n_classes=5)      # the recipe frame's class: 3 stages, BatchNorm, GAP
```

1. Flips, rotations (a close-up has no fixed "up"), crops keeping most of the frame: safe. Hue shifts are not: they can turn a sunflower's yellow into a colour no sunflower has. Brightness and contrast jitter are fine.
2. Three stages of two blocks, widths 32, 64, 128, a pool after each: $96 \to 48 \to 24 \to 12$; GAP to 128 numbers, `Linear(128, 5)`: about 288,000 parameters. A `Flatten` head would add 92,000 for nothing.
3. A 0.31 gap is overfitting: first augmentation (free, attacks the cause), then dropout before the `Linear` and weight decay (Day 17); keep the best validation epoch. **Check:** training accuracy drops below 0.99 and validation rises; if both fall, the augmentation is too strong.

# CNNs: **Limitations & What Comes Next**

# Limitations

- **Augmentation cannot invent what is missing.** It makes versions of the photos you have, not a class, viewpoint or season nobody photographed. When the test set differs in kind, real data or pretrained features (Week 5) are the fix.
- **Deep from scratch needs data and time.** ResNet and VGG trained on 1.2 million photos for days on GPUs. On a few thousand photos a 3-stage VGG-style net is the ceiling, and a pretrained network usually beats it.
- **BatchNorm wants batches.** Below about 16 images its statistics are noise; in `eval()` it uses running averages, so a model evaluated in training mode gives different, worse numbers.
- **Skip connections need matching shapes.** $F(x) + x$ only adds when the shapes agree; across a pool or a width change the skip gets a $1 \times 1$ convolution, one more thing to size correctly.

The common mistake

Augmenting the validation or test images, or evaluating without `model.eval()`: both make the validation score a lottery and the chosen epoch a wrong one. Two transforms, two modes, always.

# What Comes Next

- **Tomorrow (Day 24), detection and segmentation:** today's stacks as a *backbone* with a different head: boxes and class scores at every position (YOLO), or a label at every pixel with an upward path (U-Net), scored with IoU and mAP.
- **Day 25, self-study:** kernels, padding and shape debugging in depth, and an image-classification run end to end.
- **Day 26, self-study:** YOLO, U-Net, IoU and mAP with numbers.
- **Day 27, Olympiad practice:** augmentation techniques (mixup, cutmix, test-time augmentation), the strength dial read from the curves, and the vision habits that win points.
- **Week 5:** the networks of today's ladder, already trained on ImageNet, reused on your photos: transfer learning.

# CNNs: **Summary**

# Summary

- **The recipe:** blocks of conv $3 \times 3$, `BatchNorm2d`, ReLU; stages of blocks with a pool between them and widths doubling; a head of global average pooling and one `Linear`. Two $3 \times 3$ convs see $5 \times 5$ with fewer weights than one $5 \times 5$; after three stages a unit sees $36 \times 36$ pixels.
- **The ladder:** LeNet-5 (1998, 60 thousand parameters) set the shape; AlexNet (2012, 16.4%) added ReLU, dropout, GPUs and augmentation; VGG (2014, 7.3%) made everything $3 \times 3$ and deep; ResNet (2015, 3.57%) made 152 layers trainable with skip connections and BatchNorm.
- **BatchNorm2d** standardises each channel over the batch and its positions, two parameters per channel: 14 times fewer steps, larger learning rates, two modes to respect.
- **The skip connection** $y = F(x) + x$ lets a block be invisible and carries the gradient straight back: deeper is never worse.
- **Global average pooling** turns $(C, H, W)$ into $C$ numbers: a 390-parameter head instead of 24,582, and any image size.
- **Augmentation** applies random, label-preserving changes inside the training transform only; it narrows the gap at the price of a harder training task and more epochs. Flips and crops for most photos, no hue shifts on colour-defined classes, never on validation or test.

# CNNs: **References & Resources**

# References & Resources


[1] LeCun, Y., Bottou, L., Bengio, Y. and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE* 86(11). LeNet-5 (Fig. 2, redrawn), its parameter count and MNIST errors with and without distortions.

[2] Krizhevsky, A., Sutskever, I. and Hinton, G. (2012). ImageNet classification with deep convolutional neural networks. *NeurIPS* 25. AlexNet (Fig. 2 reused), ReLU, dropout, the 2,048-fold crop-and-flip augmentation and the overfitting remark of Section 4.1.

[3] Simonyan, K. and Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. *ICLR*. VGG: $3 \times 3$ kernels, 16 to 19 layers, configuration D (Table 1), 7.3% top-5 (Table 7).

[4] Szegedy, C. et al. (2015). Going deeper with convolutions. *CVPR*. GoogLeNet, 22 layers, 6.7% top-5 at ILSVRC 2014.

[5] He, K., Zhang, X., Ren, S. and Sun, J. (2016). Deep residual learning for image recognition. *CVPR*. The degradation problem (Fig. 1), the residual block (Fig. 2), 152 layers, 3.57% top-5. The degradation sketch on the slide is Stanford CS231n's redrawing of Fig. 1.

[6] Ioffe, S. and Szegedy, C. (2015). Batch normalization: accelerating deep network training by reducing internal covariate shift. *ICML*. 14 times fewer steps, higher learning rates, less dropout.

[7] Lin, M., Chen, Q. and Yan, S. (2014). Network in network. *ICLR*. Global average pooling as the classifier head.

# References & Resources (cont.)


[8] Russakovsky, O. et al. (2015). ImageNet large scale visual recognition challenge. *IJCV* 115. The yearly winners' errors (2010 to 2014) and the human estimate of about 5.1%.

[9] Howard, A. et al. (2017). MobileNets: efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861. Tan, M. and Le, Q. (2019). EfficientNet: rethinking model scaling for convolutional neural networks. *ICML*.

[10] Shorten, C. and Khoshgoftaar, T. (2019). A survey on image data augmentation for deep learning. *Journal of Big Data* 6(60). The catalogue of geometric and photometric augmentations and when each is label-safe.

[11] PyTorch and torchvision documentation: `nn.BatchNorm2d`, `nn.AdaptiveAvgPool2d`, `torchvision.transforms`, and the model tables with parameter counts (ResNet-18 11.7M, ResNet-50 25.6M, ResNet-152 60.2M, GoogLeNet 6.6M). https://pytorch.org/vision/stable/models.html

[12] Kaggle datasets used today, all through `kagglehub`: *Intel Image Classification* (`puneet6060/intel-image-classification`, Lab 1), *Garbage Classification* (`asdasdasasdas/garbage-classification`, TrashNet, the augmentation figures and Lab 2), *Flowers Recognition* (`alxmamaev/flowers-recognition`, the practice task). MNIST via OpenML for the label-safety figure.

[13] Figures without another source: `slides-builder/scripts/day23_cnn_figures.py` (deterministic augmentation illustrations from the public data above). No model was trained for this deck; every comparison number is quoted from the papers above.